

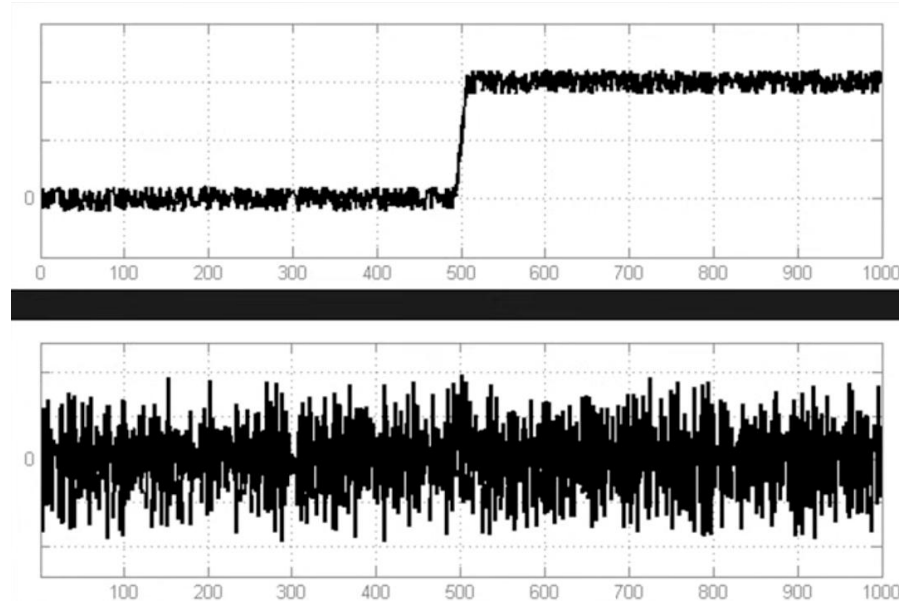
18-799: Applied Computer Vision

Spring 2026

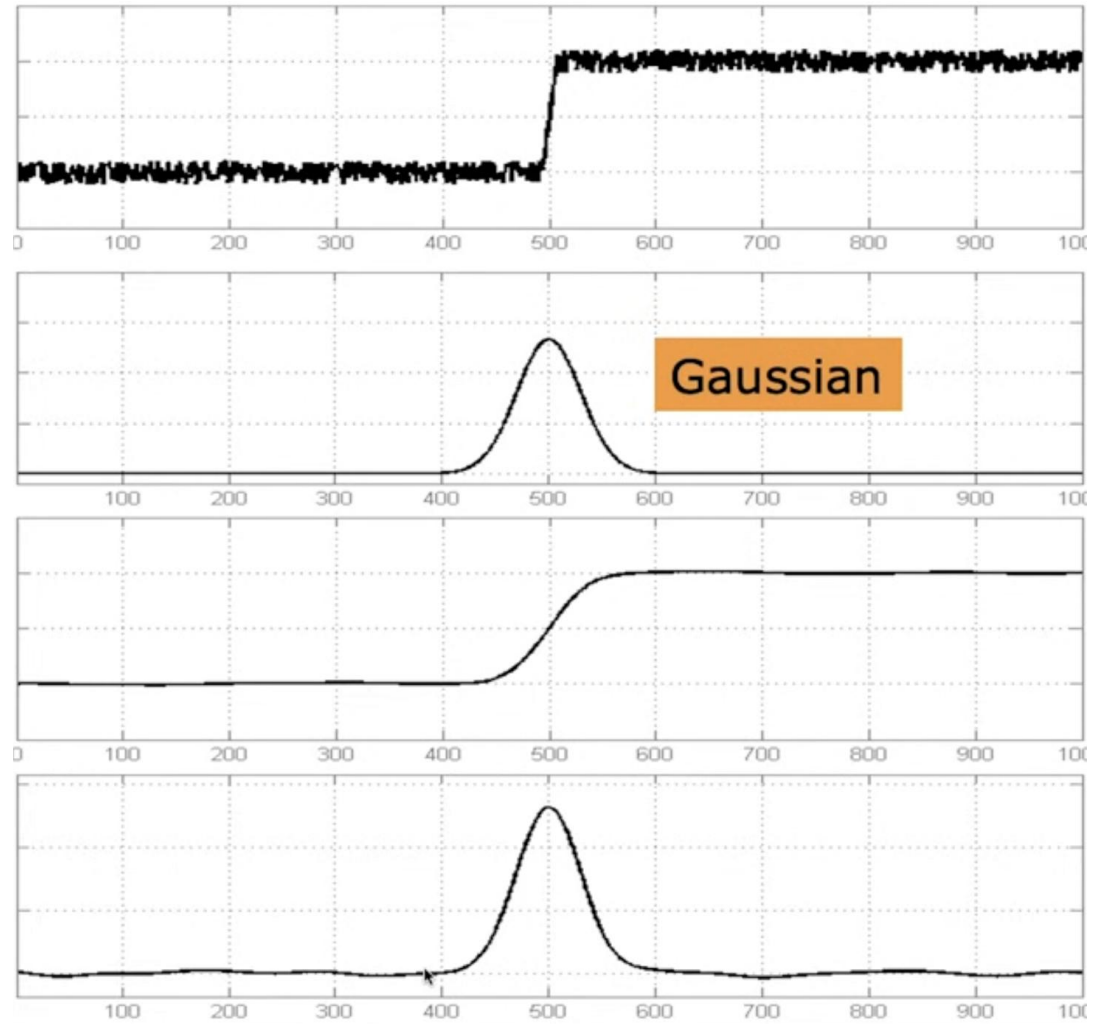
Feature Description, Matching
& Tracking

Laplacian (∇^2) Operator

- Second derivatives amplify noise even more than gradients.



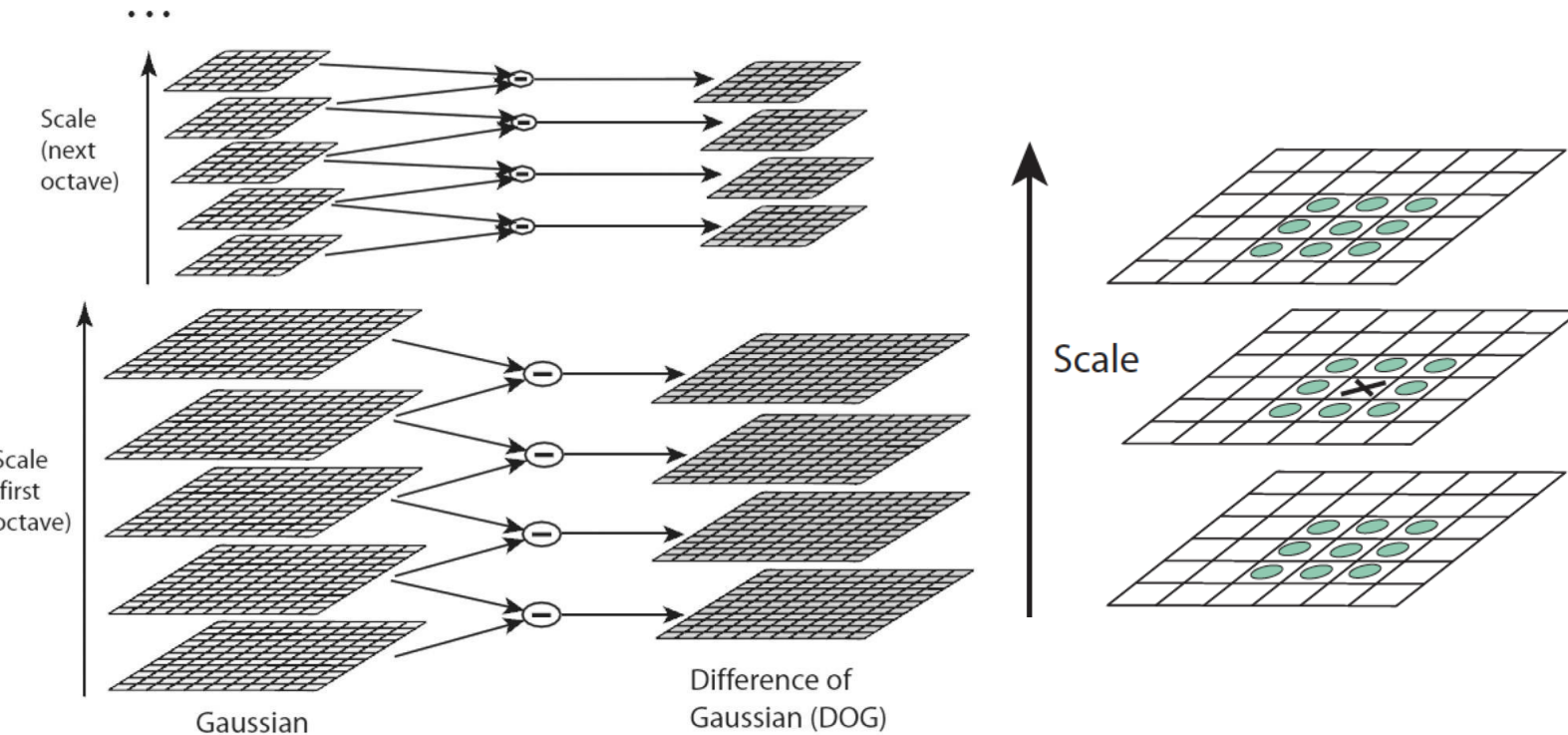
Laplacian (∇^2) Operator



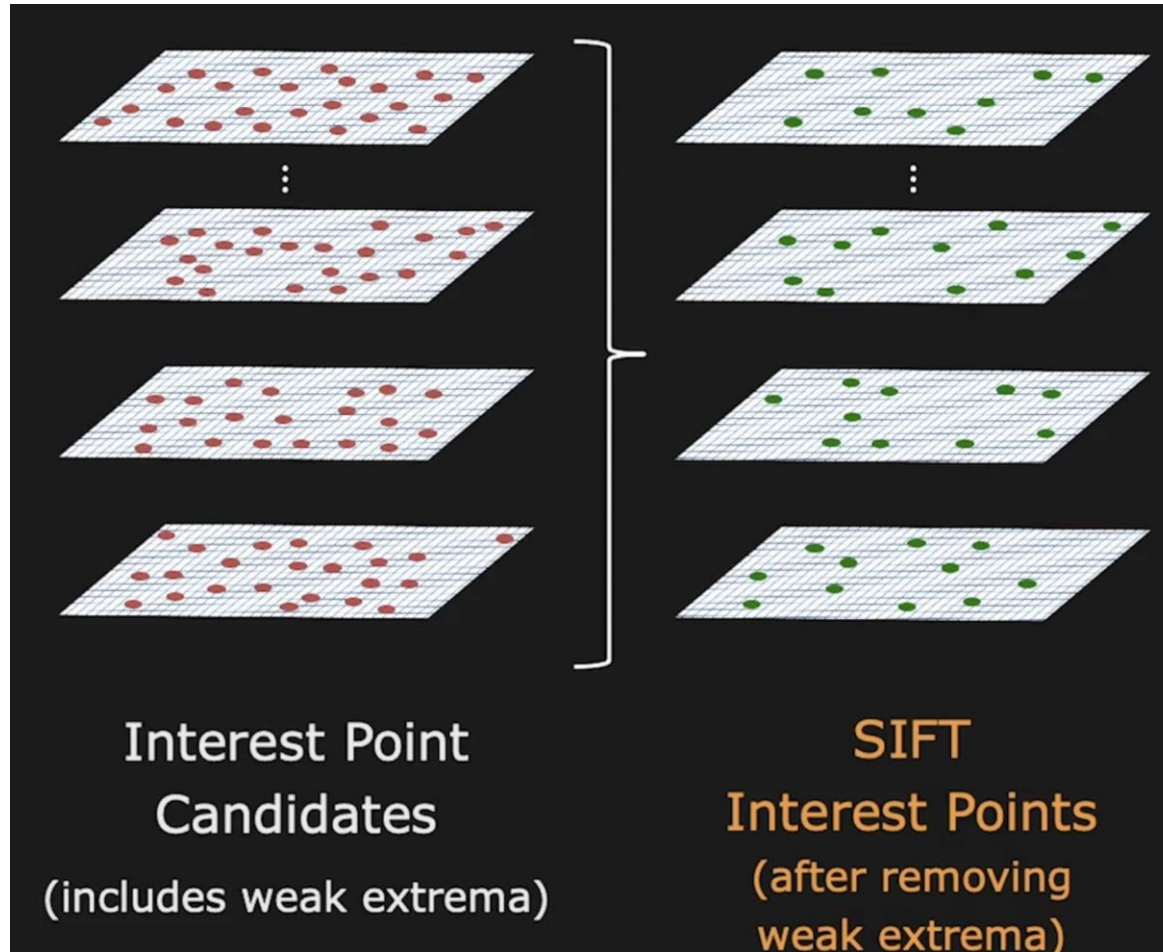
Laplacian (∇^2) Operator

- Second derivatives amplify noise even more than gradients.
- So in practice:
 - Smooth first (Gaussian)
 - Then apply Laplacian
- This leads to the Laplacian of Gaussian (LoG):
$$\nabla^2(G_\sigma * I)$$

SIFT Feature Detection



SIFT Feature Detection



Orientation Assignment (Rotation Invariance)

- We need to assign a canonical orientation to each keypoint to achieve rotation invariance.
 - Step 1 - Use Image at Keypoint Scale
$$L(x, y, \sigma^*) = G_{\sigma^*} * I(x, y)$$
 - We work at the detected scale σ^* .
 - Step 2 - Compute Gradients

$$L_x, L_y$$
$$m(x, y) = \sqrt{L_x^2 + L_y^2}$$
$$\theta(x, y) = \tan^{-1}(L_y/L_x)$$

Orientation Assignment (Rotation Invariance)

- Step 3 - Build Orientation Histogram

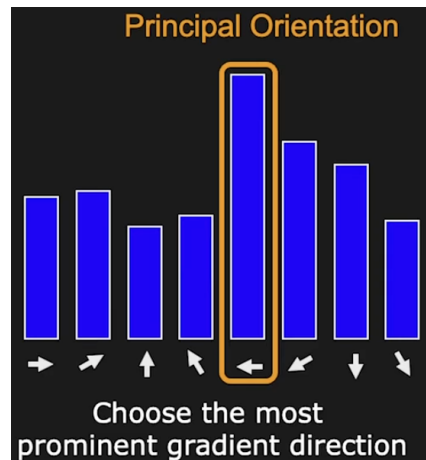
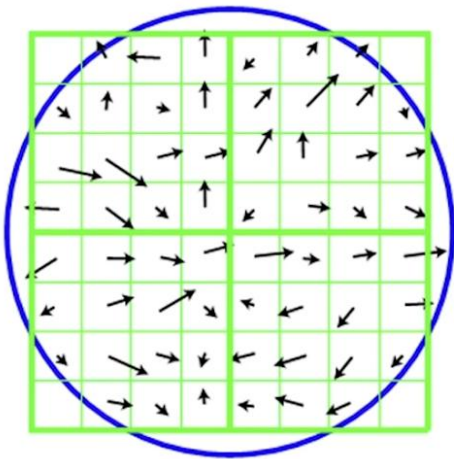
- Neighborhood radius $\propto \sigma^*$
- 36 bins (10° each)
- Each pixel votes:

$$H(\theta) += m(x, y) \cdot w(x, y)$$

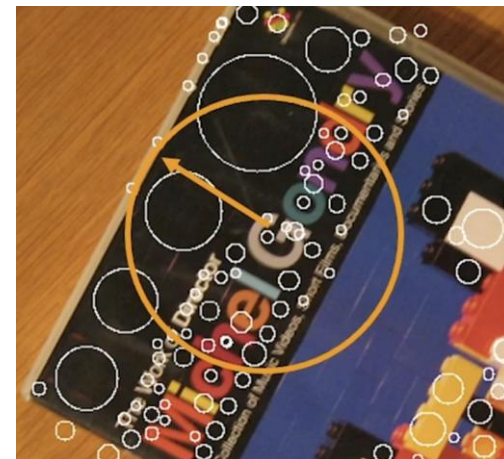
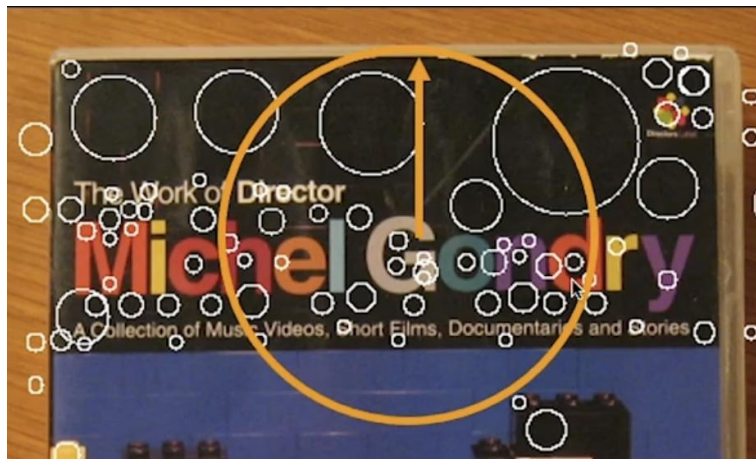
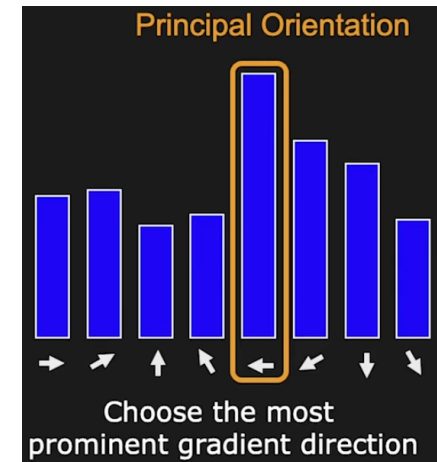
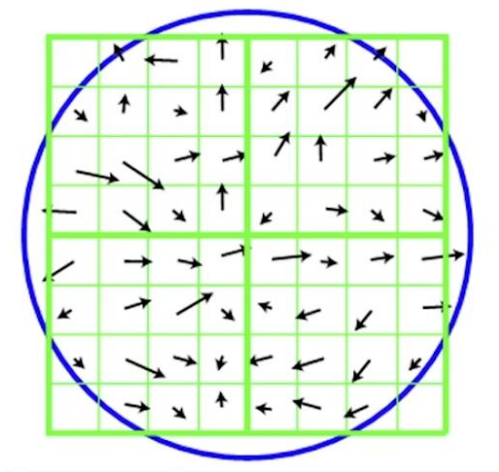
- where $w(x, y)$ is Gaussian weighting

- Step 4 - Select Dominant Orientation

$$\theta^* = \arg \max H(\theta)$$



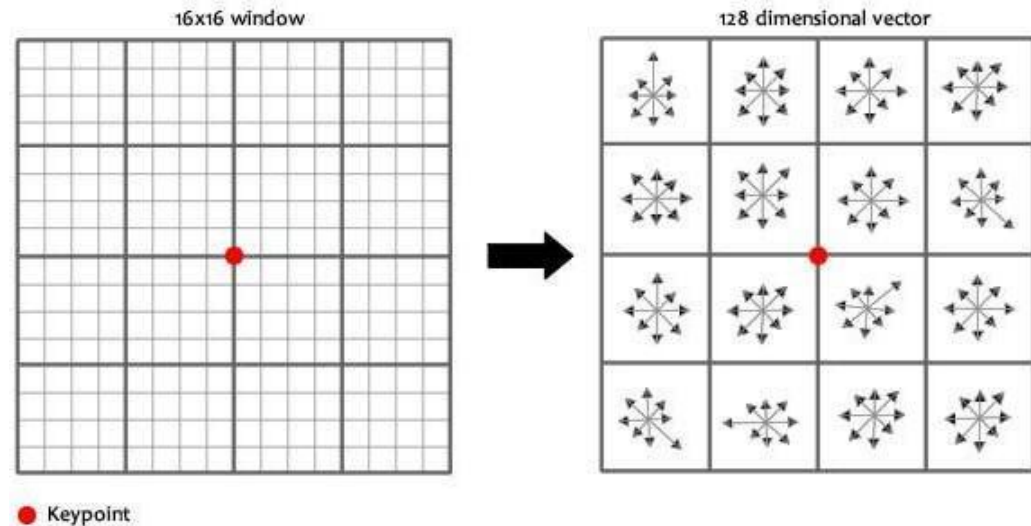
Orientation Assignment (Rotation Invariance)



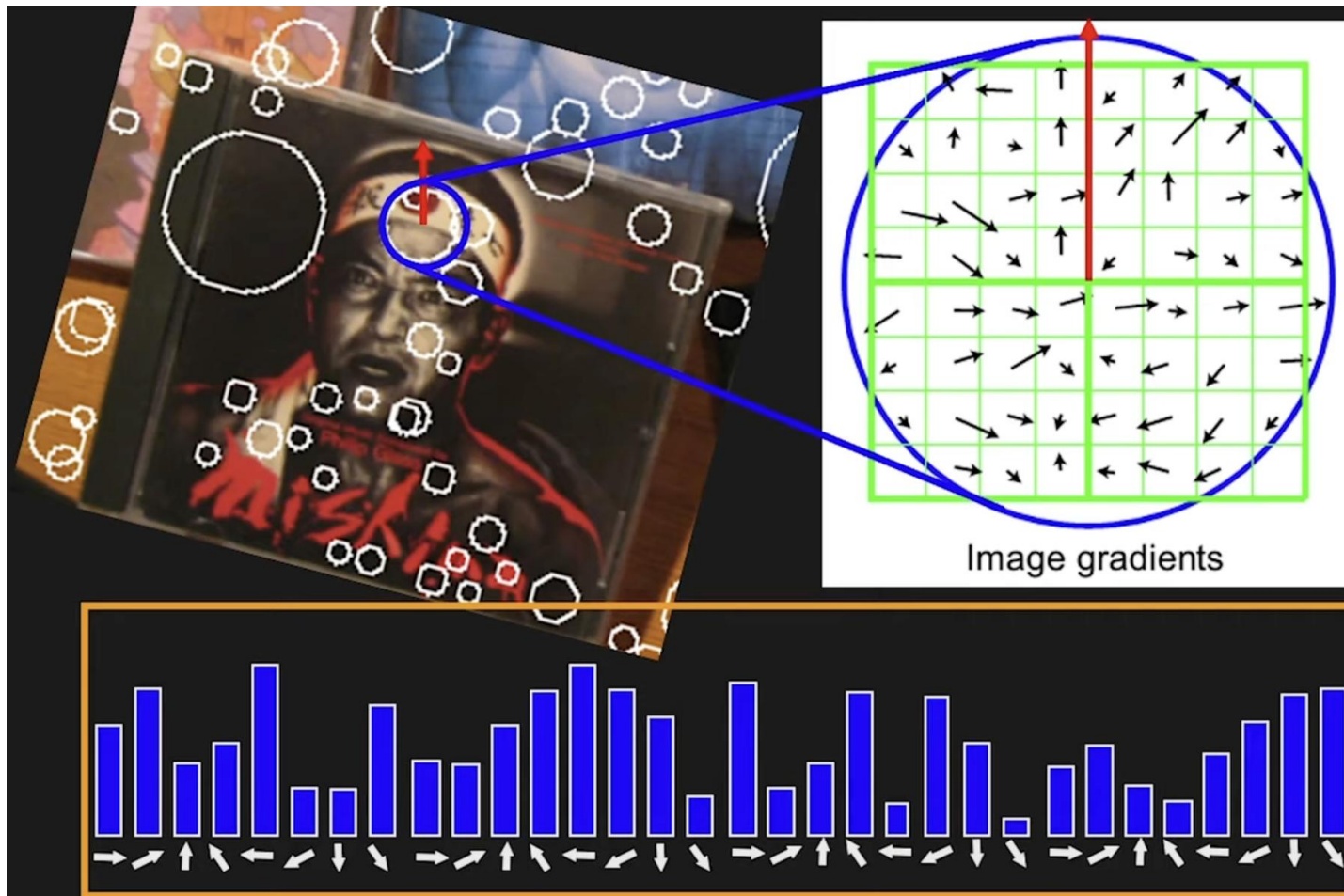
Feature Descriptors

Scale Invariant Feature Transform (SIFT)

- SIFT is **invariance to image scale** and rotation.
- Features formed by **computing the gradient at each pixel in a 16x16 window around the detected keypoint**, using the appropriate level of the Gaussian pyramid at which the keypoint was detected.



SIFT Descriptor



Feature Matching

Euclidean distance

- Euclidean(vector magnitude) distances in feature space is used for ranking potential matches

$$d(\mathbf{u}, \mathbf{v}) = \left(\sum_i (u_i - v_i)^2 \right)^{1/2}$$

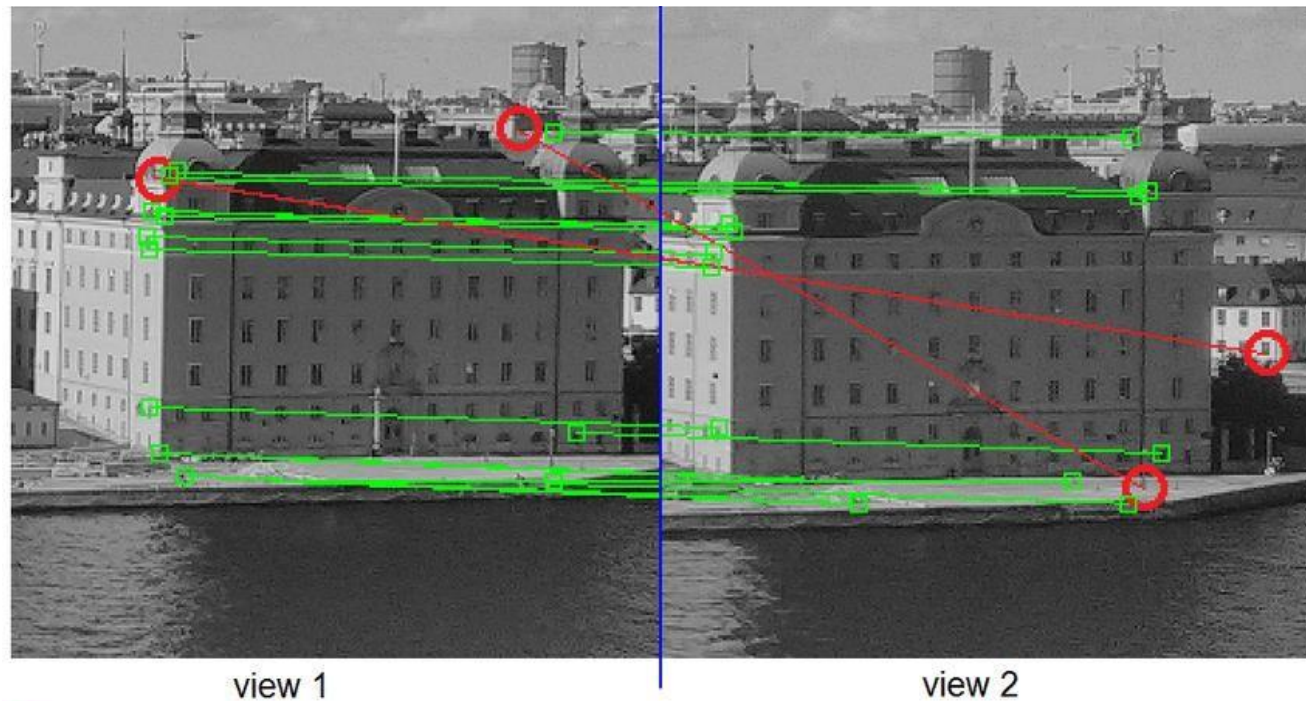
Several possible matching strategies:

- Return all feature vectors with **d smaller than a threshold**.
- Nearest neighbor: **feature vector with smallest d**.
- Nearest neighbor distance ratio:

$$\text{NNDR} = \frac{d_1}{d_2}$$

- d1, d2: distances to the nearest and 2nd nearest neighbors.
- If NNDR is small, nearest neighbor is a good match.

Feature Matching



○ outliers □ correct matches

Feature Matching

Recall → • true positive rate (TPR),

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{P};$$

• false positive rate (FPR),

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{\text{FP}}{N};$$

Precision → • positive predictive value (PPV),

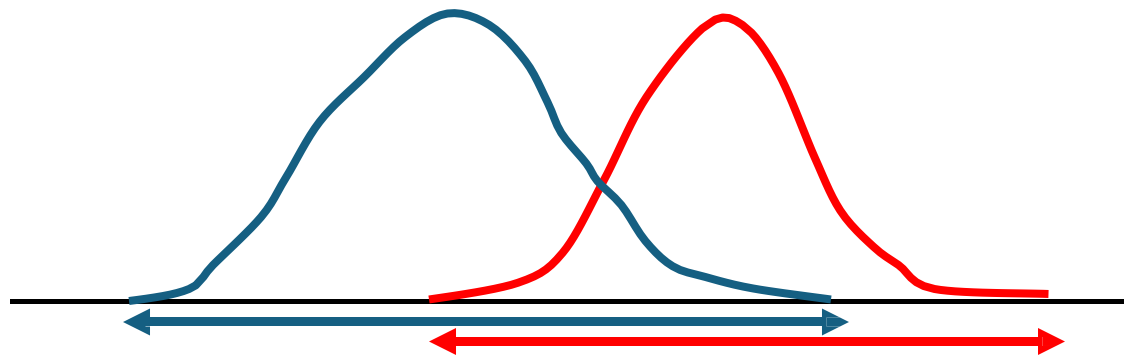
$$\text{PPV} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{\text{TP}}{P'};$$

• accuracy (ACC),

$$\text{ACC} = \frac{\text{TP} + \text{TN}}{P + N}.$$

$$\text{F-score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

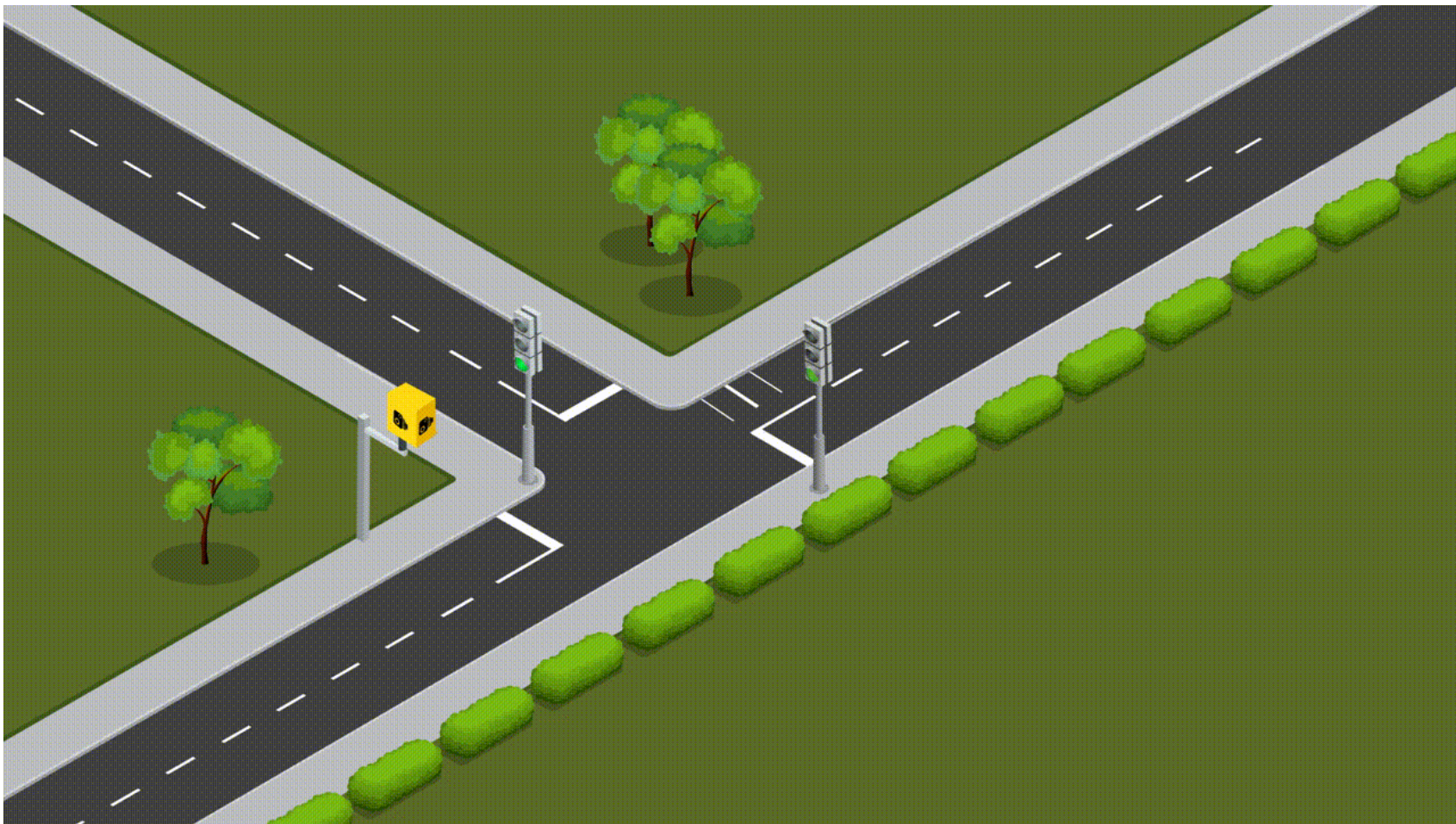
Example



Test Result

Courtesy to Darlene Goldstein, 2003

Speed camera



Optical Mouse



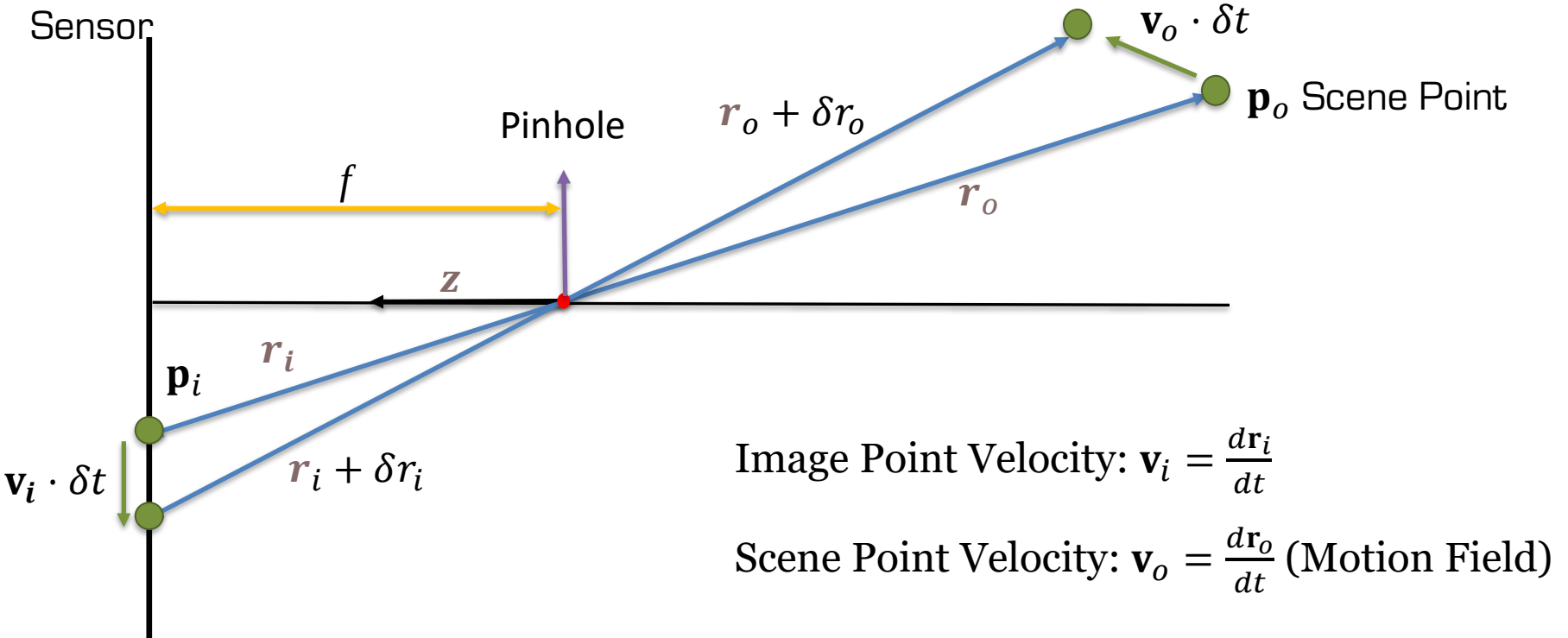
Video Quality

- Determine intermediate frame
- Image stabilization

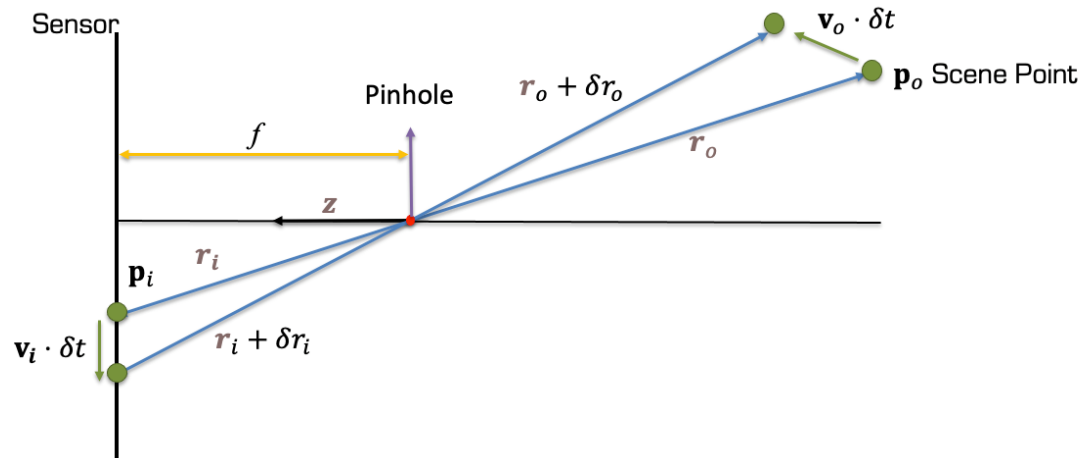
Motion field

- A representation of the movement of objects or features within a visual scene:
 - describe the direction and magnitude of motion at each point in an image or video sequence.
 - They can be represented as vectors, where each vector corresponds to the motion at a specific location.

Motion field



Motion field



Perspective projection: $\frac{\mathbf{r}_i}{f} = \frac{\mathbf{r}_o}{\mathbf{r}_o \cdot \mathbf{z}}$

Image Point Velocity: $\mathbf{v}_i = \frac{d\mathbf{r}_i}{dt} = f \frac{(\mathbf{r}_o \cdot \mathbf{z})\mathbf{v}_o - (\mathbf{v}_o \cdot \mathbf{z})\mathbf{r}_o}{(\mathbf{r}_o \cdot \mathbf{z})^2}$

Motion field

- How to obtain motion field?
 - Trackers
 - Approximate the motion field using optical flows

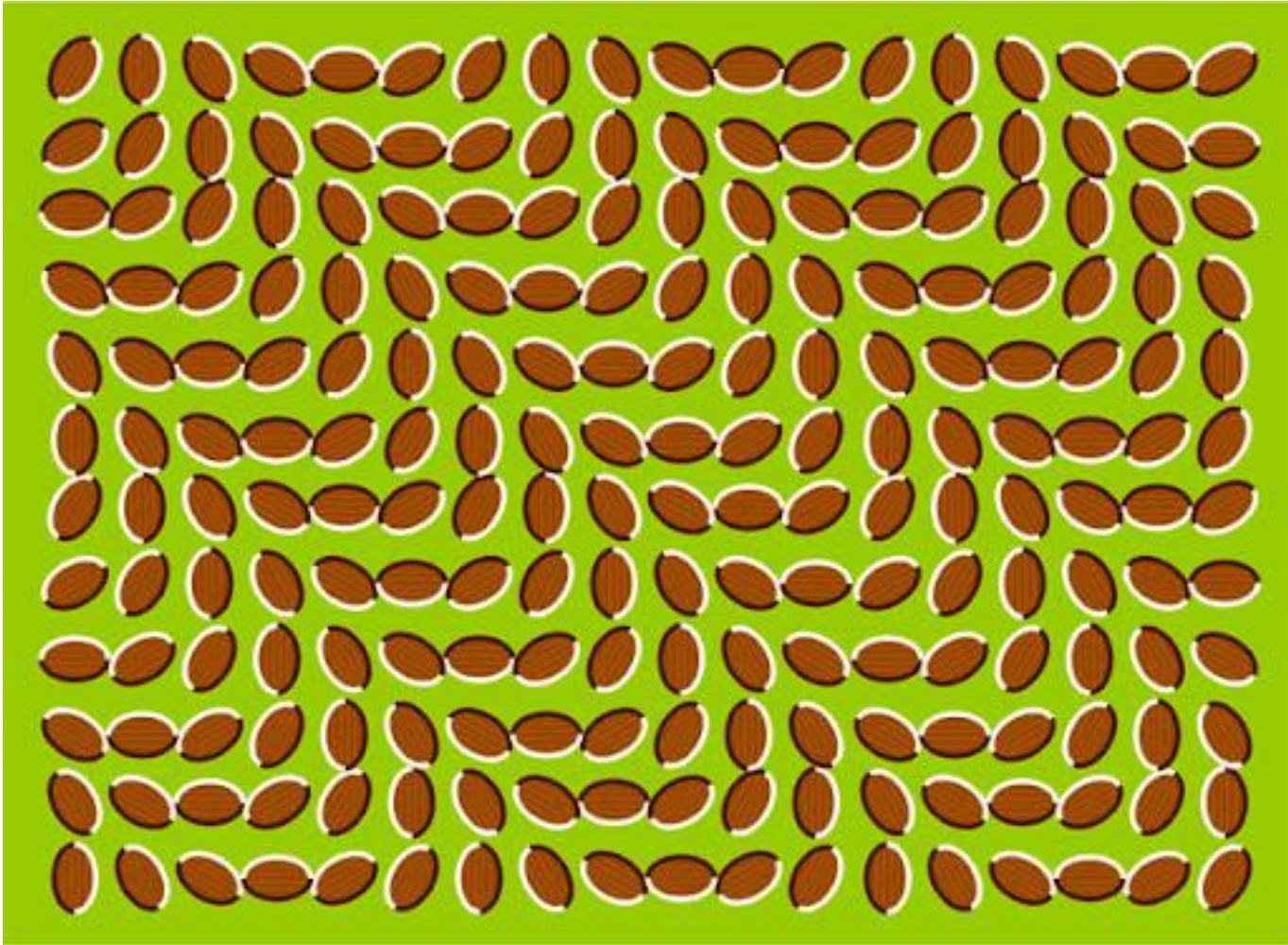
Optical flow



Optical flow



Optical flow

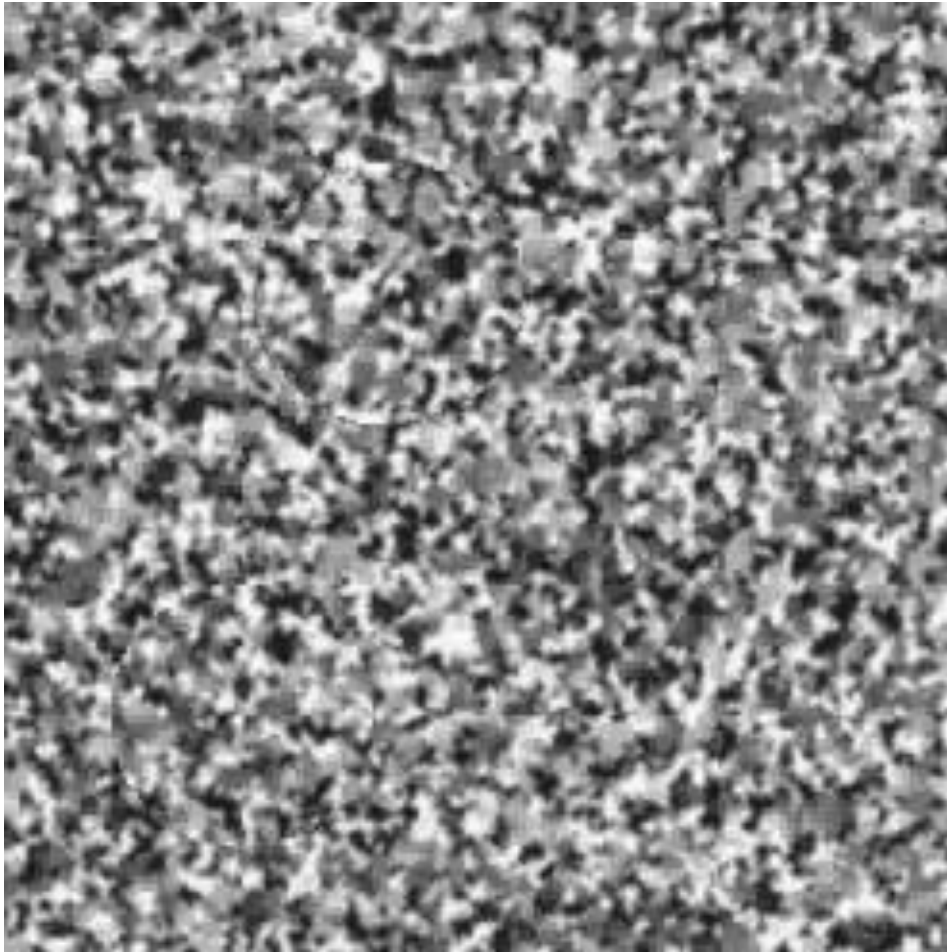


Optical flow

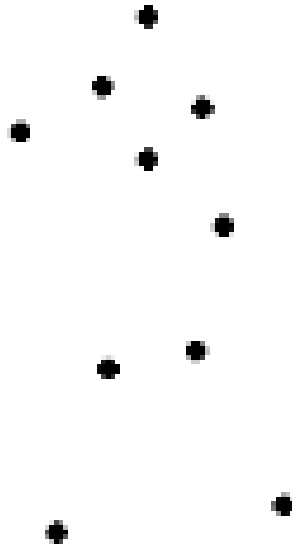




Motion is a powerful perceptual cue

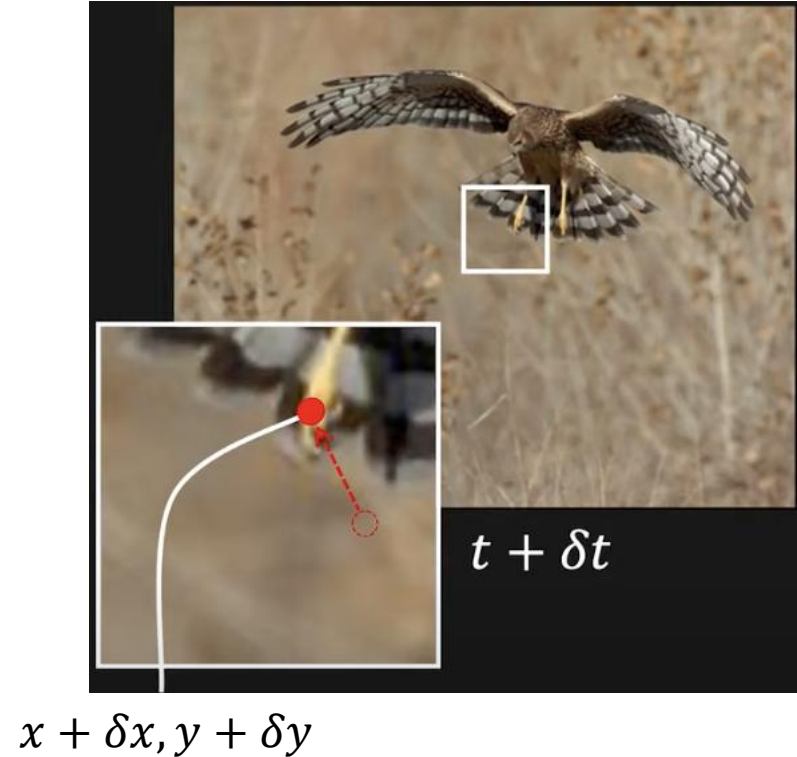
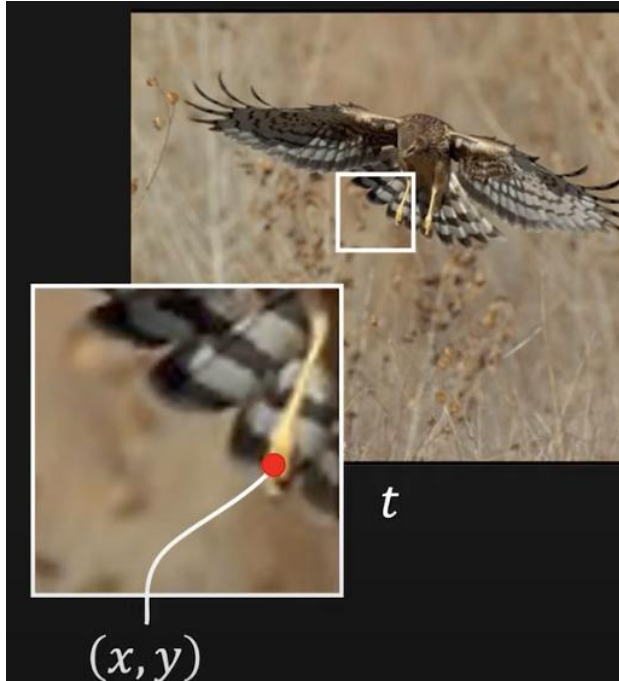


Motion is a powerful perceptual cue



- G. Johansson, "Visual Perception of Biological Motion and a Model For Its Analysis", *Perception and Psychophysics* 14, 201-211, 1973.

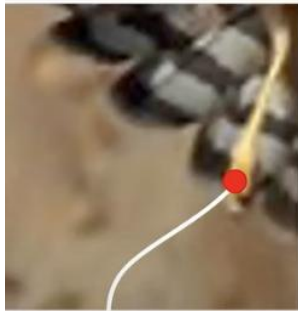
Optical flow Constraint Equation



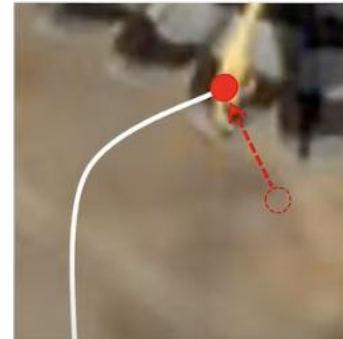
Displacement: $(\delta x, \delta y)$

Optical Flow: $(u, v) = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t} \right)$

Optical flow Constraint Equation



$I(x, y, t)$



$I(x + \delta x, y + \delta y, t + \delta t)$

Assumption #1:

- Brightness of image point remains constant over time

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$$

Assumption #2:

- Displacement $(\delta x, \delta y)$ and time step δt are small

Taylor Series Expansion

- Expand a function as an infinite sum of its derivatives

$$f(x + \delta x) = f(x) + \frac{\partial f}{\partial x} \delta x + \frac{\partial^2 f}{\partial x^2} \frac{\delta x^2}{2!} + \dots + \frac{\partial^n f}{\partial x^n} \frac{\delta x^n}{n!}$$

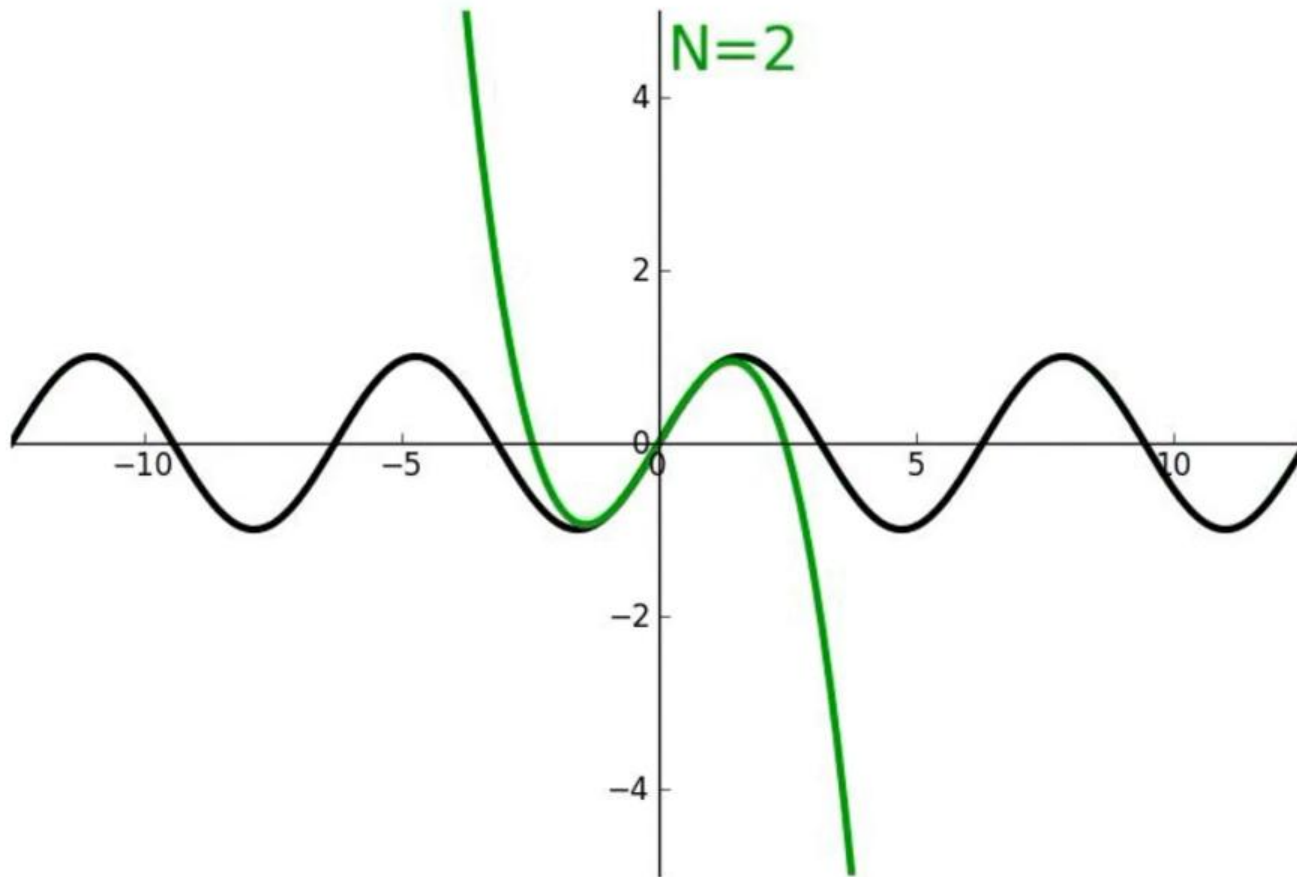
- If δx is small:

$$f(x + \delta x) = f(x) + \frac{\partial f}{\partial x} \delta x + O(\delta x^2) \rightarrow \text{Almost Zero}$$

- For a function of three variables with small $\delta x, \delta y, \delta t$:

$$f(x + \delta x, y + \delta y, t + \delta t) \approx f(x, y, t) + \frac{\partial f}{\partial x} \delta x + \frac{\partial f}{\partial y} \delta y + \frac{\partial f}{\partial t} \delta t$$

Taylor Series Expansion



Optical flow Constraint Equation

- Assumption #2:
 - Displacement $(\delta x, \delta y)$ and time step δt are small

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + \frac{\partial I}{\partial x} \delta x + \frac{\partial I}{\partial y} \delta y + \frac{\partial I}{\partial t} \delta t$$

$$\Rightarrow I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + I_x \delta x + I_y \delta y + I_t \delta t$$

Optical flow Constraint Equation

The constraints

$$(1) \dots I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$$

$$(2) \dots I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + I_x \delta x + I_y \delta y + I_t \delta t$$

- Subtract [1] from [2]: $I_x \delta x + I_y \delta y + I_t \delta t = 0$

Divide by δt and take limit as $\delta t \rightarrow 0$: $I_x \frac{\partial x}{\partial t} + I_y \frac{\partial y}{\partial t} + I_t = 0$

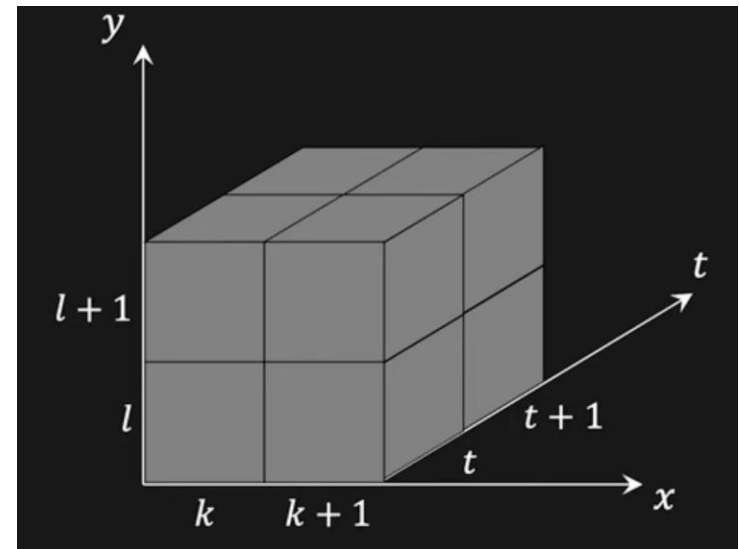
-

Constraint Equation: $I_x u + I_y v + I_t = 0$ (u, v) : Optical Flow

Optical flow Constraint Equation

Computing partial derivatives:

$$I_x(k, l, t) = \frac{1}{4} [I(k+1, l, t) + I(k+1, l, t+1) + I(k+1, l+1, t) + I(k+1, l+1, t+1)] - \frac{1}{4} [I(k, l, t) + I(k, l, t+1) + I(k, l+1, t) + I(k, l+1, t+1)]$$



Optical flow Constraint Equation

Direction of Normal Flow:

Unit vector perpendicular to the constraint line:

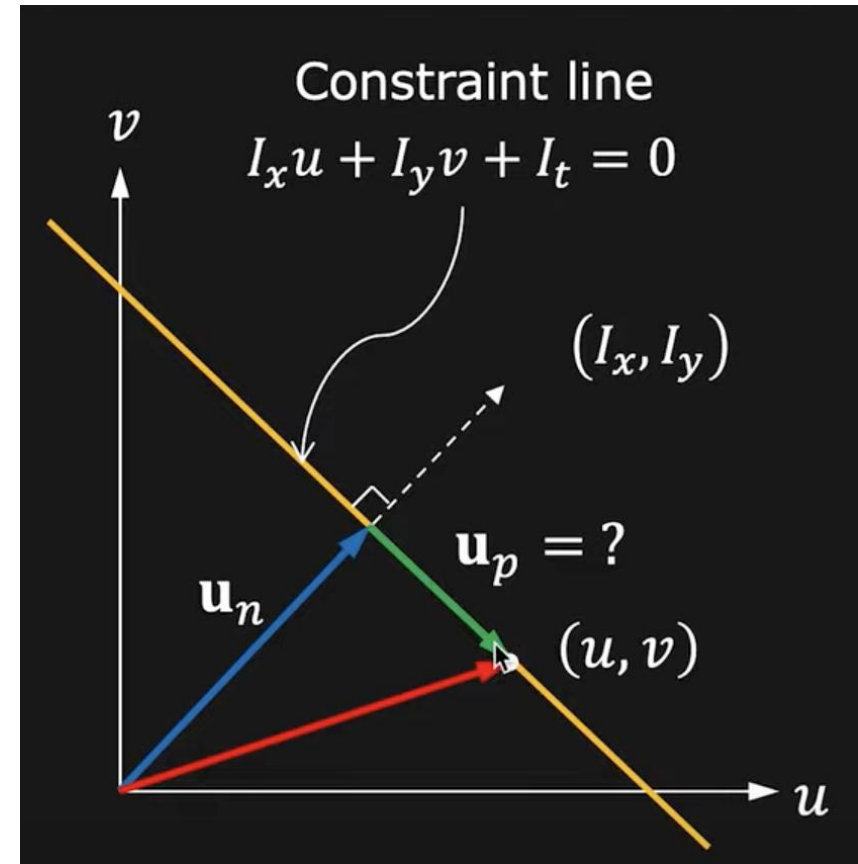
$$\hat{\mathbf{u}}_n = \frac{(I_x, I_y)}{\sqrt{I_x^2 + I_y^2}}$$

Magnitude of Normal Flow:

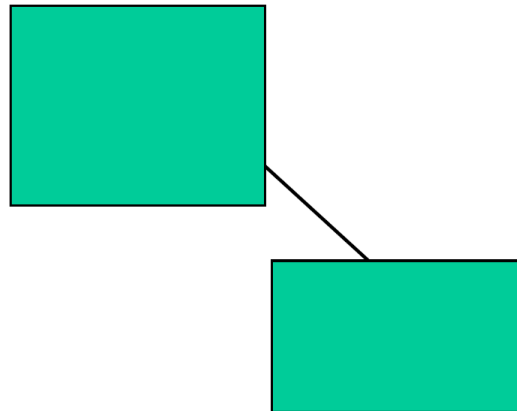
Distance of origin from the constraint line:

$$|\mathbf{u}_n| = \frac{|I_t|}{\sqrt{I_x^2 + I_y^2}}$$

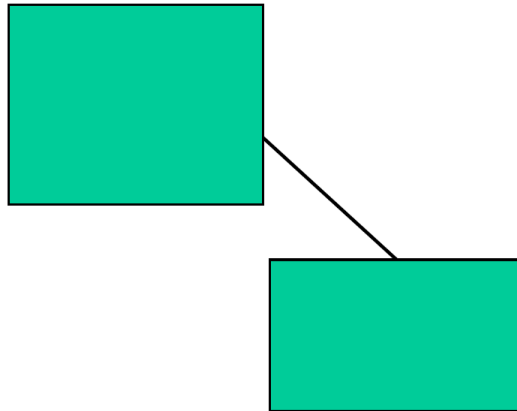
$$\mathbf{u}_n = \frac{|I_t|}{(I_x^2 + I_y^2)} (I_x, I_y)$$



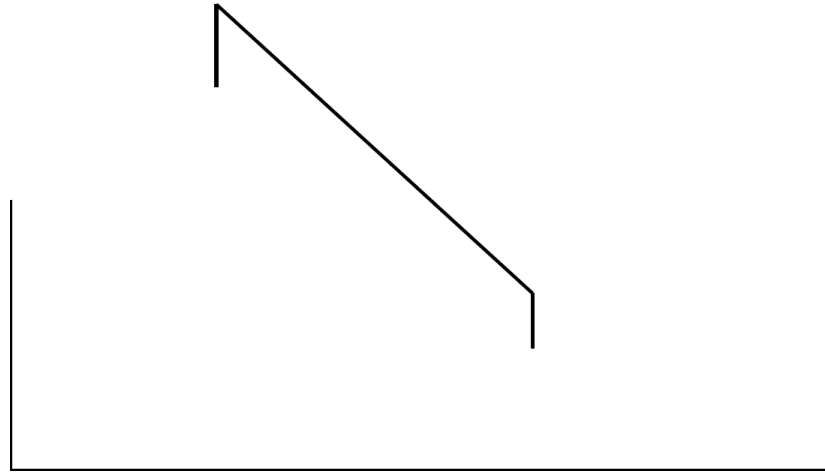
Aperture Problem



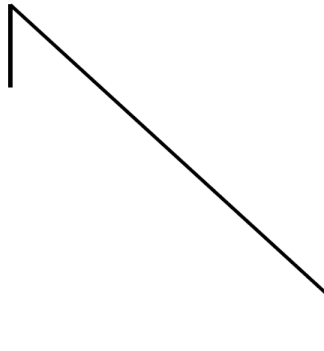
Aperture Problem



Aparture Problem



Aperture Problem



Lucas-Kanade Solution



Assumption:

- For each pixel, assume Motion Field, and hence Optical Flow (u, v) , is constant within a small neighborhood W .

Lucas-Kanade Solution



For all points $(k, l) \in W: I_x(k, l)u + I_y(k, l)v + I_t(k, l) = 0$

Let the size of window W be $n \times n$

In matrix form:

$$\begin{bmatrix} I_x(1,1) & I_y(1,1) \\ I_x(k,l) & I_y(k,l) \\ \vdots & \vdots \\ I_x(n,n) & I_y(n,n) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} I_t(1,1) \\ I_t(k,l) \\ \vdots \\ I_t(n,n) \end{bmatrix}$$

Lucas-Kanade Solution



For all points $(k, l) \in W: I_x(k, l)u + I_y(k, l)v + I_t(k, l) = 0$

Let the size of window W be $n \times n$

In matrix form:

$$\begin{bmatrix} I_x(1,1) & I_y(1,1) \\ I_x(k,l) & I_y(k,l) \\ \vdots & \vdots \\ I_x(n,n) & I_y(n,n) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} I_t(1,1) \\ I_t(k,l) \\ \vdots \\ I_t(n,n) \end{bmatrix}$$

Lucas-Kanade Solution



Solve linear system: $A\mathbf{u} = B$

$$A^T A \mathbf{u} = A^T B$$

(Least-Squares using Pseudo-Inverse)

Lucas-Kanade Solution

- When Does Optical Flow Estimation Work?

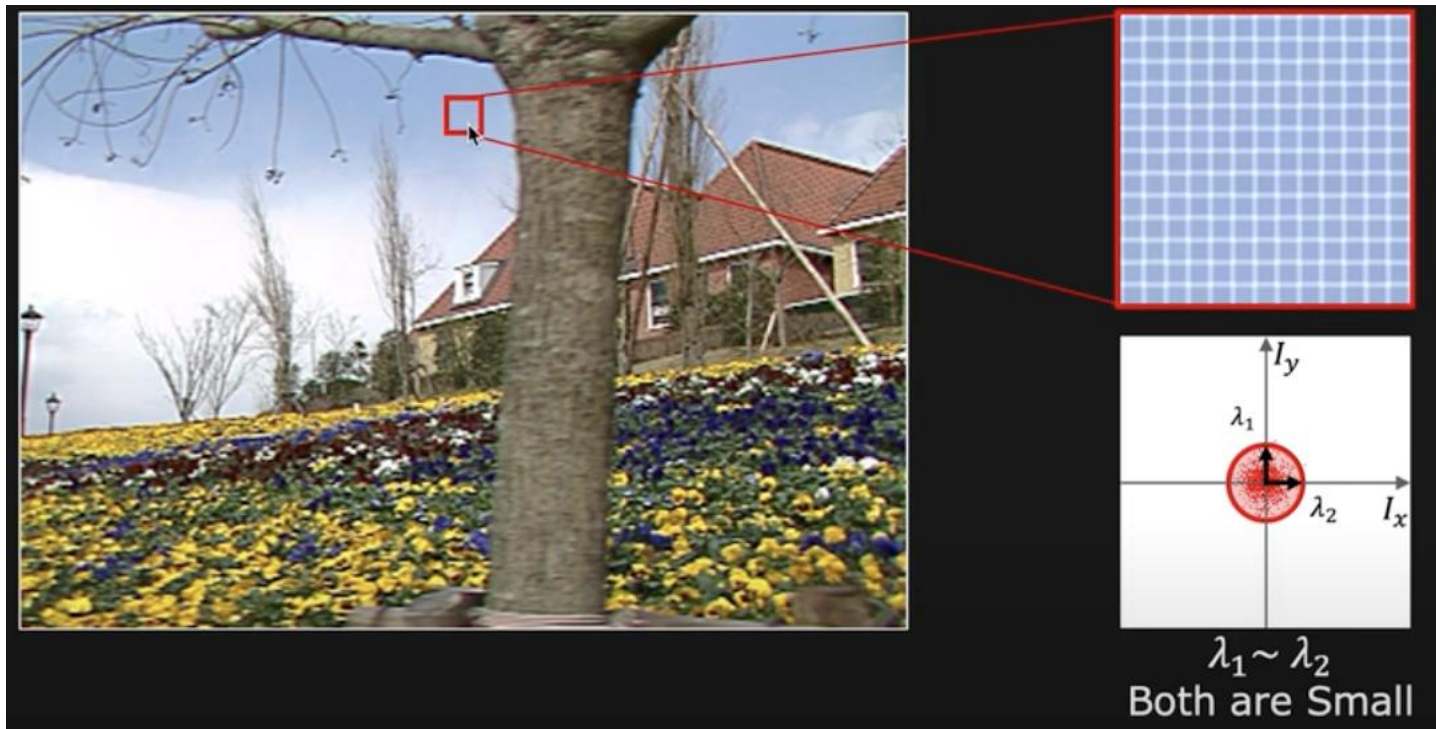
$$A\mathbf{u} = B$$

$$A^T A \mathbf{u} = A^T B$$

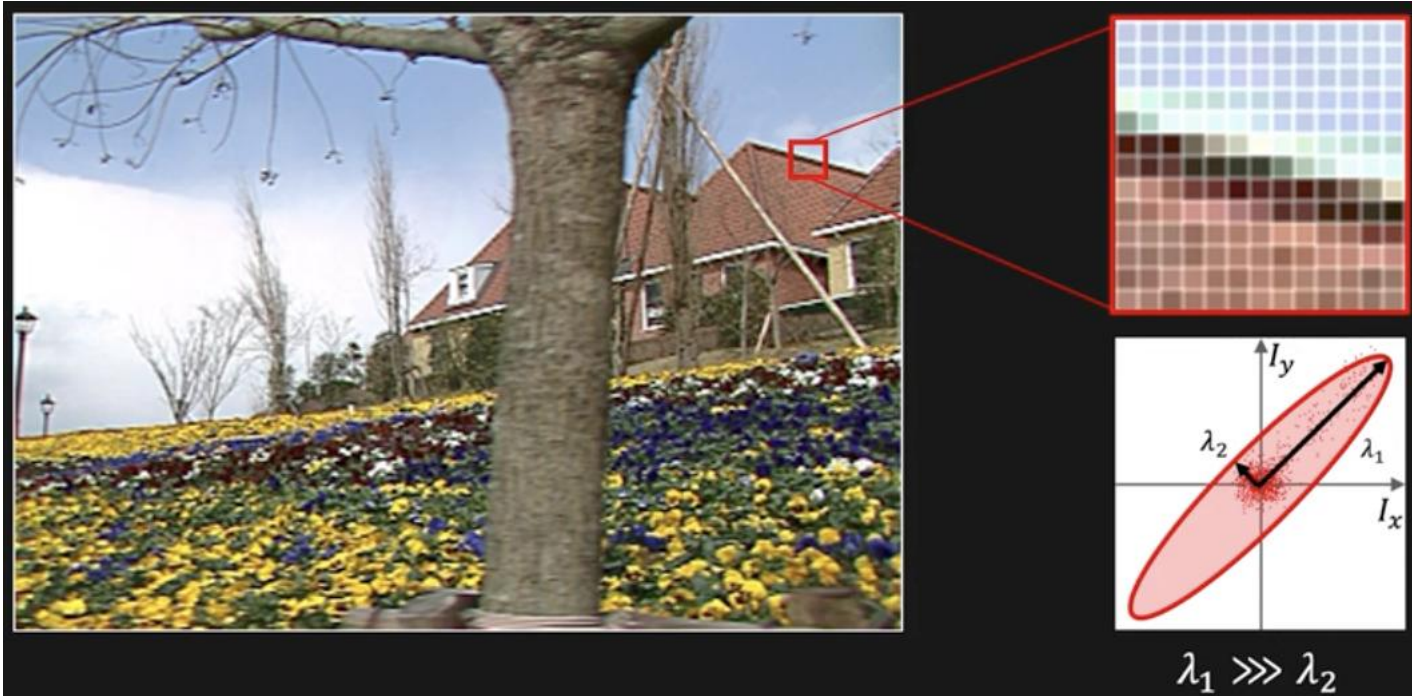
- $A^T A$ must be invertible. That is $\det(A^T A) \neq 0$
- $A^T A$ must be well-conditioned.
- If λ_1 and λ_2 are eigen values of $A^T A$, then

$$\begin{aligned} \lambda_1 &> \epsilon \text{ and } \lambda_2 > \epsilon \\ \lambda_1 &\geq \lambda_2 \text{ but not } \lambda_1 \gg \lambda_2 \end{aligned}$$

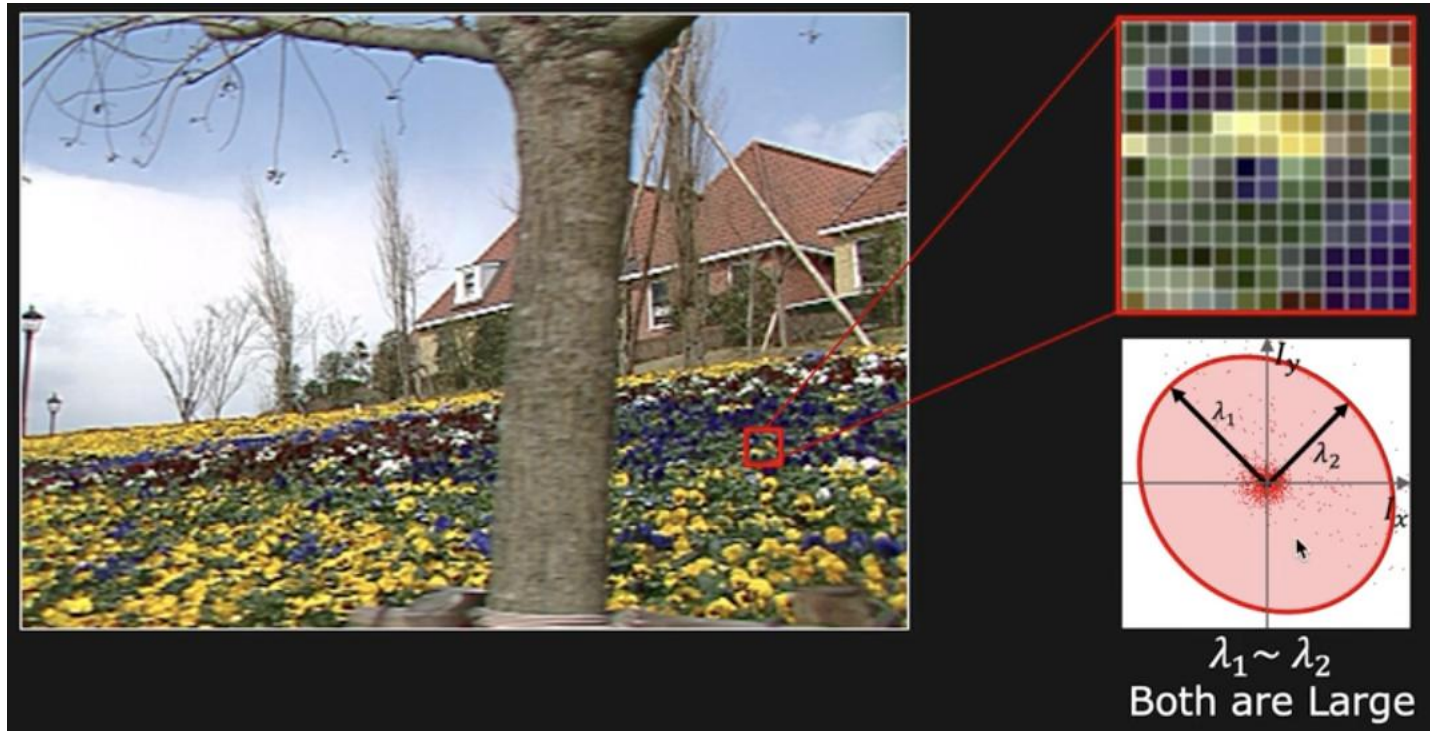
Lucas-Kanade Solution



Lucas-Kanade Solution



Lucas-Kanade Solution

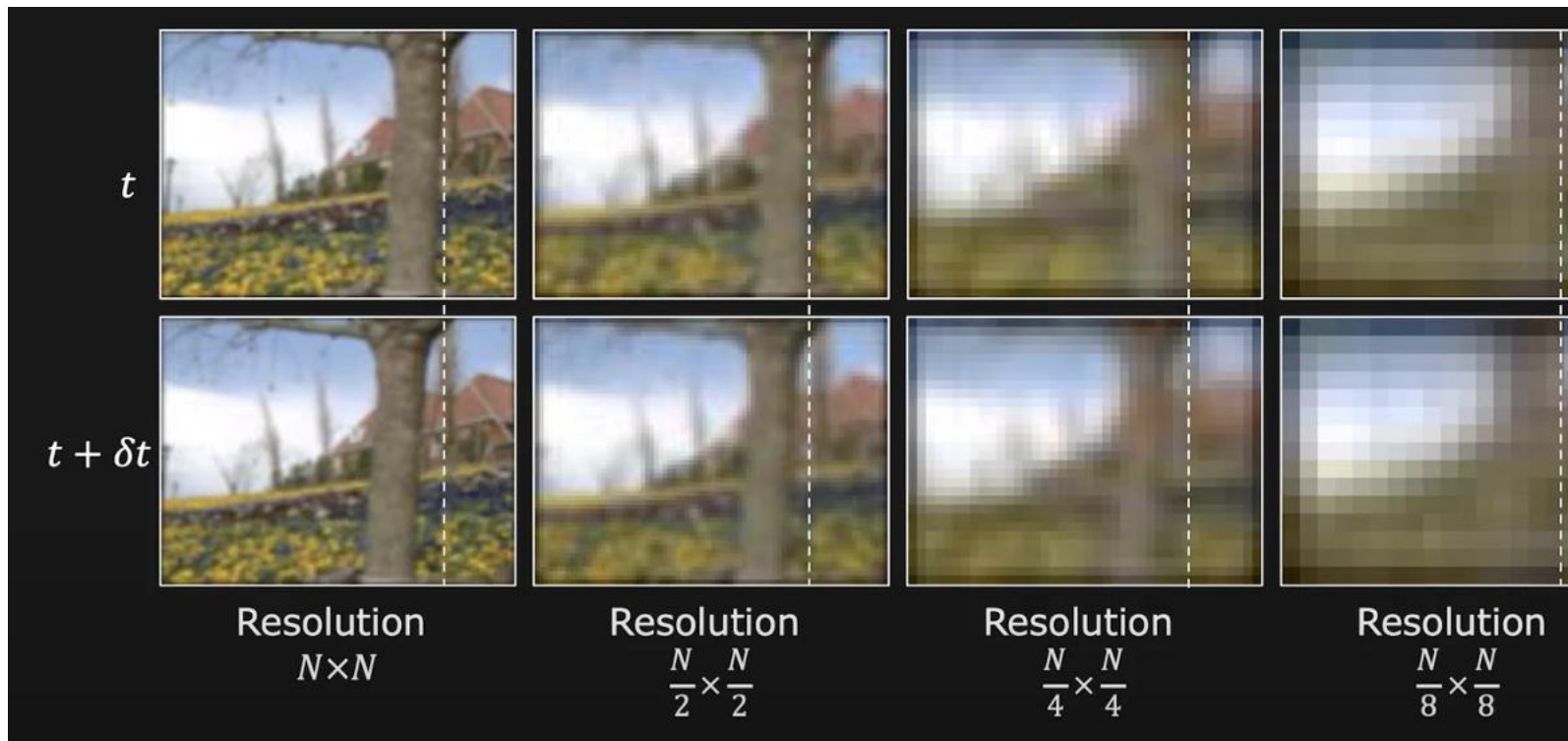


Large Motion

- Taylor Series approximation is not valid

Coarse-to-Fine Estimation Algorithm

- Taylor Series approximation is not valid



Coarse-to-Fine Estimation Algorithm

